# Lecture Notes on NAND Flash Memory Technology

Chapter 6: Error Correction and Controllers

## Chapter 6. Error Correction and Controllers

## 1 6.1 ECC Fundamentals for NAND

### • Error Characteristics in NAND

**Definition 1.1** (NAND Error Types)**.**

- **Raw Bit Errors**: $10^{-4}$ to $10^{-2}$ (end of life)
- **Clustered Errors**: Adjacent cell failures
- **Random Errors**: Distributed single-bit failures
- **Burst Errors**: Word line or bit line failures

Error correction capability requirement:

$$t_{correct} \geq \lceil -\log_2(\text{BER}_{target}) \cdot n \rceil \tag{1}$$

Code rate optimization:

$$R = \frac{k}{n} = \frac{\text{Information bits}}{\text{Total bits}} \tag{2}$$

### • Hamming Distance

Minimum Hamming distance:

$$d_{min} = t_{detect} + t_{correct} + 1 \tag{3}$$

For $t$-error correction:

$$d_{min} \geq 2t + 1 \tag{4}$$

## 2 6.2 BCH Codes Implementation

### • BCH Code Mathematics

**Theorem 2.1** (BCH Generator Polynomial)**.** For $t$-error correcting BCH code over GF($2^m$):

$$g(x) = \text{LCM}[m_1(x), m_3(x), ..., m_{2t-1}(x)] \tag{5}$$

where $m_i(x)$ is the minimal polynomial of $\alpha^i$.

Syndrome calculation:

$$S_j = r(\alpha^j) = \sum_{i=0}^{n-1} r_i \alpha^{ji} \tag{6}$$

Error locator polynomial:

$$\Lambda(x) = \prod_{i=1}^{v} (1 - X_i x) \tag{7}$$

### • BCH Decoding Algorithm

Berlekamp-Massey algorithm:

1. Calculate syndrome: $S = [S_1, S_2, ..., S_{2t}]$
2. Find error locator: $\Lambda(x)$
3. Calculate error positions: roots of $\Lambda(x)$
4. Correct errors

Computational complexity:

$$\text{Operations} = O(t^2) \text{ for } t\text{-error correction} \tag{8}$$

### • Multi-level BCH

Extended BCH for MLC:

$$\text{Capability}_{\text{total}} = \sum_{i=1}^{n_{bits}} t_i \tag{9}$$

Unequal error protection:

$$t_{MSB} > t_{CSB} > t_{LSB} \tag{10}$$

## 3 6.3 LDPC for Modern NAND

### • LDPC Code Structure

**Definition 3.1** (Low-Density Parity-Check Matrix)**.** LDPC code defined by sparse parity-check matrix $H$:

- Low density: few 1's per row/column
- Regular: constant row/column weight
- Irregular: variable row/column weight

Tanner graph representation:

$$\text{Variable nodes} \leftrightarrow \text{Check nodes} \tag{11}$$

Code rate:

$$R = 1 - \frac{\text{rank}(H)}{n} \approx 1 - \frac{m}{n} \tag{12}$$

### • Belief Propagation Decoding

Message passing algorithm:

$$m_{v\to c}^{(l+1)} = \sum_{c' \in N(v)\setminus c} m_{c'\to v}^{(l)} \tag{13}$$

$$m_{c\to v}^{(l+1)} = 2\tanh^{-1}\left(\prod_{v' \in N(c)\setminus v} \tanh\left(\frac{m_{v'\to c}^{(l)}}{2}\right)\right) \tag{14}$$

Log-likelihood ratio (LLR):

$$\text{LLR} = \ln\left(\frac{P(\text{bit}=0|y)}{P(\text{bit}=1|y)}\right) \tag{15}$$

• **LDPC Performance**

Decoding threshold:

$$\sigma^* = \sqrt{\frac{2R}{E_b/N_0}} \tag{16}$$

Minimum distance scaling:

$$d_{min} = \Theta(n^{1-1/r}) \tag{17}$$

# 4 6.4 Flash Translation Layer (FTL) Design

• **Address Translation**

**Theorem 4.1** (Logical to Physical Mapping)**.** Address translation function:

$$\text{PBA} = f(\text{LBA}, \text{Mapping table}) \tag{18}$$

Mapping granularity trade-off:

$$\text{Page-level}: \quad \text{RAM} = \frac{N_{pages} \times \text{Address size}}{8} \tag{19}$$

$$\text{Block-level}: \quad \text{RAM} = \frac{N_{blocks} \times \text{Address size}}{8} \tag{20}$$

• **Mapping Table Management**

Cached mapping table:

$$\text{Hit ratio} = \frac{\text{Cache hits}}{\text{Total accesses}} \tag{21}$$

Update frequency:

$$f_{update} = \frac{\text{Write IOPS}}{\text{Page size / Sector size}} \tag{22}$$

• **Metadata Management**

Metadata overhead:

$$\text{Overhead} = \frac{\text{Metadata size}}{\text{User data size}} \times 100\% \tag{23}$$

Typical metadata:

- Logical address: 4-8 bytes
- ECC syndrome: 100-200 bytes/page
- Timestamp: 4-8 bytes
- Status flags: 1-2 bytes

# 5 6.5 Wear Leveling Algorithms

## • Static Wear Leveling

**Definition 5.1** (Wear Leveling Objective)**.** Minimize maximum erase count difference:

$$\min \max_{i,j} |EC_i - EC_j| \tag{24}$$

where $EC_i$ is erase count of block $i$.

Perfect wear leveling bound:

$$EC_{max} - EC_{min} \leq 1 \tag{25}$$

## • Dynamic Wear Leveling

Selection algorithms:

1. **Round-robin**: Sequential block selection
2. **Least-worn**: Select minimum erase count
3. **Random**: Probabilistic selection

Wear leveling effectiveness:

$$\eta_{WL} = \frac{\text{Theoretical lifetime}}{\text{Actual lifetime}} \tag{26}$$

## • Advanced Wear Leveling

Temperature-aware wear leveling:

$$\text{Priority} = f(EC, T_{avg}, \text{Access pattern}) \tag{27}$$

Adaptive algorithms:

$$\text{Trigger threshold} = \alpha \cdot \frac{\text{Total writes}}{\text{Total blocks}} \tag{28}$$

# 6 6.6 Garbage Collection

## • GC Trigger Conditions

**Theorem 6.1** (GC Necessity)**.** Garbage collection required when:

$$\frac{N_{free}}{N_{total}} < \text{Threshold}_{GC} \tag{29}$$

Typically: $\text{Threshold}_{GC}$ = 10-15%

Write amplification:

$$\text{WA} = \frac{\text{Data written to NAND}}{\text{Data written by host}} \tag{30}$$

• **Victim Block Selection**

Selection criteria:

$$\text{Cost} = \alpha \cdot N_{valid} + \beta \cdot EC + \gamma \cdot \text{Age} \tag{31}$$

Greedy algorithm:

$$\text{Block}_{victim} = \arg\min_i \left( \frac{N_{valid,i}}{N_{total,i}} \right) \tag{32}$$

• **GC Performance Optimization**

Background GC scheduling:

$$t_{GC} = f(\text{Idle time}, \text{Free space}, \text{Write queue}) \tag{33}$$

Parallel GC operations:

$$\text{Throughput}_{GC} = N_{planes} \times \text{Throughput}_{single} \tag{34}$$

# 7 6.7 Bad Block Management

• **Bad Block Classification**

**Definition 7.1** (Block Health States)**.**

- **Good**: BER $< 10^{-4}$
- **Marginal**: $10^{-4} \leq$ BER $< 10^{-2}$
- **Bad**: BER $\geq 10^{-2}$
- **Reserved**: Factory-marked bad blocks

Bad block growth rate:

$$\frac{dN_{bad}}{dN_{cycle}} = A \cdot \left( \frac{N_{cycle}}{N_0} \right)^{\alpha} \tag{35}$$

• **Bad Block Detection**

Detection criteria:

$$\text{Block bad if: } \begin{cases} \text{Erase failure} \\ \text{Program failure} \\ \text{ECC uncorrectable} \\ \text{Excessive wear} \end{cases} \tag{36}$$

Proactive detection:

$$\text{Health score} = f(\text{BER}, \text{EC}, t_{program}, t_{erase}) \tag{37}$$

• **Spare Block Allocation**

Reserve pool sizing:

$$N_{spare} = N_{bad,initial} + \alpha \cdot N_{PE,spec} + \text{Margin} \tag{38}$$

Typical allocation: 2-5% of total blocks

# 8 Examples

**Example 8.1** (BCH Code Design)**.** Design BCH code for: - Page size: 4KB data - Target BER: $10^{-12}$ - Raw BER: $10^{-3}$

**Solution:** Data bits: $k = 4096 \times 8 = 32768$ bits Required error correction: $t = \frac{k \times \text{Raw BER}}{2} = 16$ errors

BCH parameters:

$$m = \lceil \log_2(k + r) \rceil = 16 \tag{39}$$

$$n \leq 2^{16} - 1 = 65535 \tag{40}$$

$$r = m \times t = 16 \times 16 = 256 \text{ bits} \tag{41}$$

Code rate: $R = \frac{32768}{32768+256} = 0.992$

**Example 8.2** (Write Amplification Calculation)**.** Calculate WA for: - Host writes: 100 GB/day - Valid data per GC: 50% - Over-provisioning: 20%

**Solution:**

$$\text{GC writes} = \text{Host writes} \times \frac{1 - \text{OP}}{1} \times \frac{\text{Valid ratio}}{1} \tag{42}$$

$$= 100 \times \frac{0.8}{1} \times \frac{0.5}{1} = 40 \text{ GB/day} \tag{43}$$

Total NAND writes = 100 + 40 = 140 GB/day Write amplification = 140/100 = 1.4

# 9 Homework Problems

**Assignment 6.1**: Error Correction and Controllers

1. **BCH Code Implementation**:
   - Design (255,239) BCH code
   - Implement Berlekamp-Massey decoder
   - Calculate computational complexity
   - Optimize for hardware implementation
2. **LDPC Performance Analysis**:
   - Design rate-0.9 LDPC code
   - Implement sum-product decoder
   - Compare with BCH performance
   - Analyze decoding convergence
3. **FTL Design**:
   - Design hybrid mapping scheme
   - Page-level for hot data, block-level for cold
   - Calculate RAM requirements
   - Optimize for random vs sequential access

4. **Wear Leveling Optimization**:
    - Implement temperature-aware algorithm
    - Model thermal cycling effects
    - Design adaptive threshold algorithm
    - Minimize write amplification
5. **Garbage Collection**:
    - Design cost-benefit GC algorithm
    - Include wear leveling in victim selection
    - Implement background GC scheduling
    - Analyze QoS impact
6. **Bad Block Prediction**:
    - Implement machine learning predictor
    - Use BER, EC, timing features
    - Design proactive migration
    - Optimize spare pool allocation
7. **Multi-stream SSD**:
    - Design stream-aware FTL
    - Separate hot/cold data streams
    - Optimize GC for each stream
    - Analyze lifetime improvement
8. **QLC ECC System**:
    - Design multi-level ECC
    - Implement soft-decision decoding
    - Use read retry information
    - Optimize for 1TB QLC SSD
9. **3D NAND Controller**:
    - Handle layer-dependent characteristics
    - Implement string-aware scheduling
    - Design interference mitigation
    - Optimize for vertical scaling
10. **AI-Enhanced Controller**:
    - Implement workload prediction
    - Design adaptive ECC strength
    - Use reinforcement learning for GC

- Optimize system-level performance

---

**Next Chapter Preview**: Chapter 7 will cover "3D NAND Technology" including vertical architectures, manufacturing challenges, and scaling advantages.